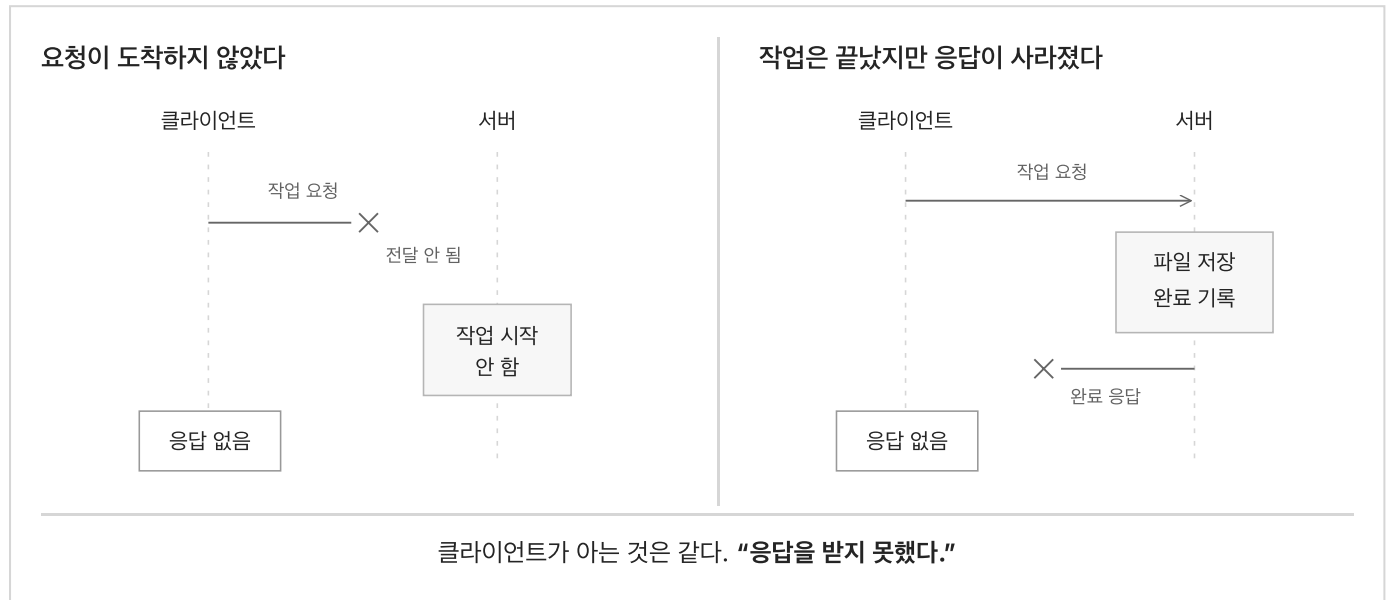


요청한 작업의 응답이 오지 않을 때, 처리 상태를 어떻게 관리할 수 있을까?

같은 타임아웃이어도 결과는 다를 수 있다

파일을 생성하는 요청을 보냈는데 응답이 오지 않았다. 다시 요청하면 될까? 서버가 아무 일도 하지 않았다면 괜찮겠지만, 이미 파일을 만들었다면 같은 작업을 한 번 더 하게 된다.



타임아웃은 기다리기를 끝냈다는 뜻이다. 작업이 실패했다는 뜻은 아니다. 서버에서는 완료됐어도 클라이언트는 결과를 모를 수 있다. 이때의 '결과 불명'은 실패가 아니라, 판단할 정보가 부족한 상태다.

먼저, 무엇을 허용하지 않을지 정한다



여기서는 파일 생성 연산이 다시 실행되는 것까지 막을 필요는 없다. 사용자에게 남는 결과를 기준으로 보면 된다.

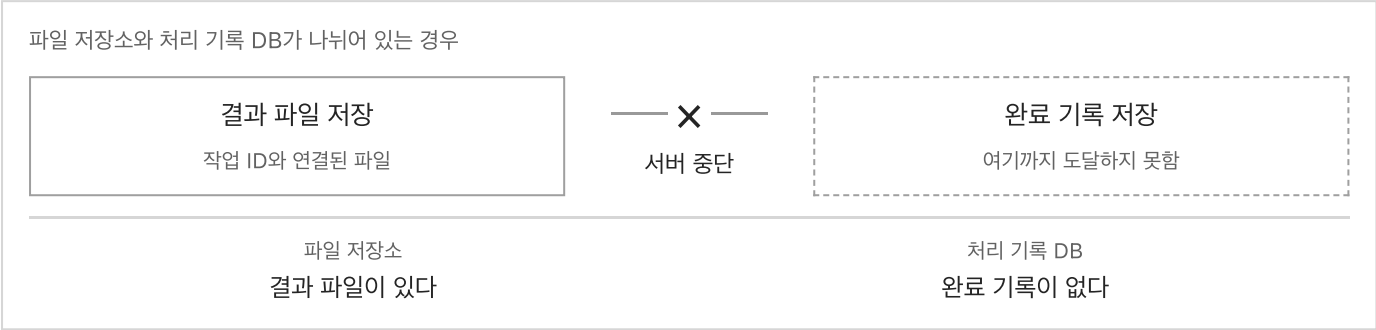
같은 작업을 여러 번 요청해도, 사용자에게는 하나의 확정된 결과만 제공한다.

한 번 확정된 결과는 재시도로 바뀌지 않아야 한다. 완료를 확인하기 전에는 성공했다고 안내하지 않고, 결과를 모른다면 재확인이나 복구로 이어갈 방법을 둔다. 이 조건을 정한 뒤에 필요한 상태를 생각해보자.

같은 작업임을 알아도, 기록이 비어 있을 수 있다

첫 요청에 작업 ID를 부여하고 재시도할 때도 같은 ID를 사용한다. 서버가 이미 확정된 결과를 찾으면 그 결과를 돌려준다. 반복 요청에도 의도한 효과가 중복으로 생기지 않게 하는 멍등성을 이 방식으로 구현할 수 있다.

다만 ID는 같은 작업을 알아보는 수단이다. 여러 실행이 겹쳐도 결과를 하나만 확정하는 규칙이 함께 있어야 한다. 그런데 그 규칙을 만들다 보면 이런 상황을 만나게 된다.



완료 기록이 없다는 것과 작업이 실행되지 않았다는 것은 다르다.

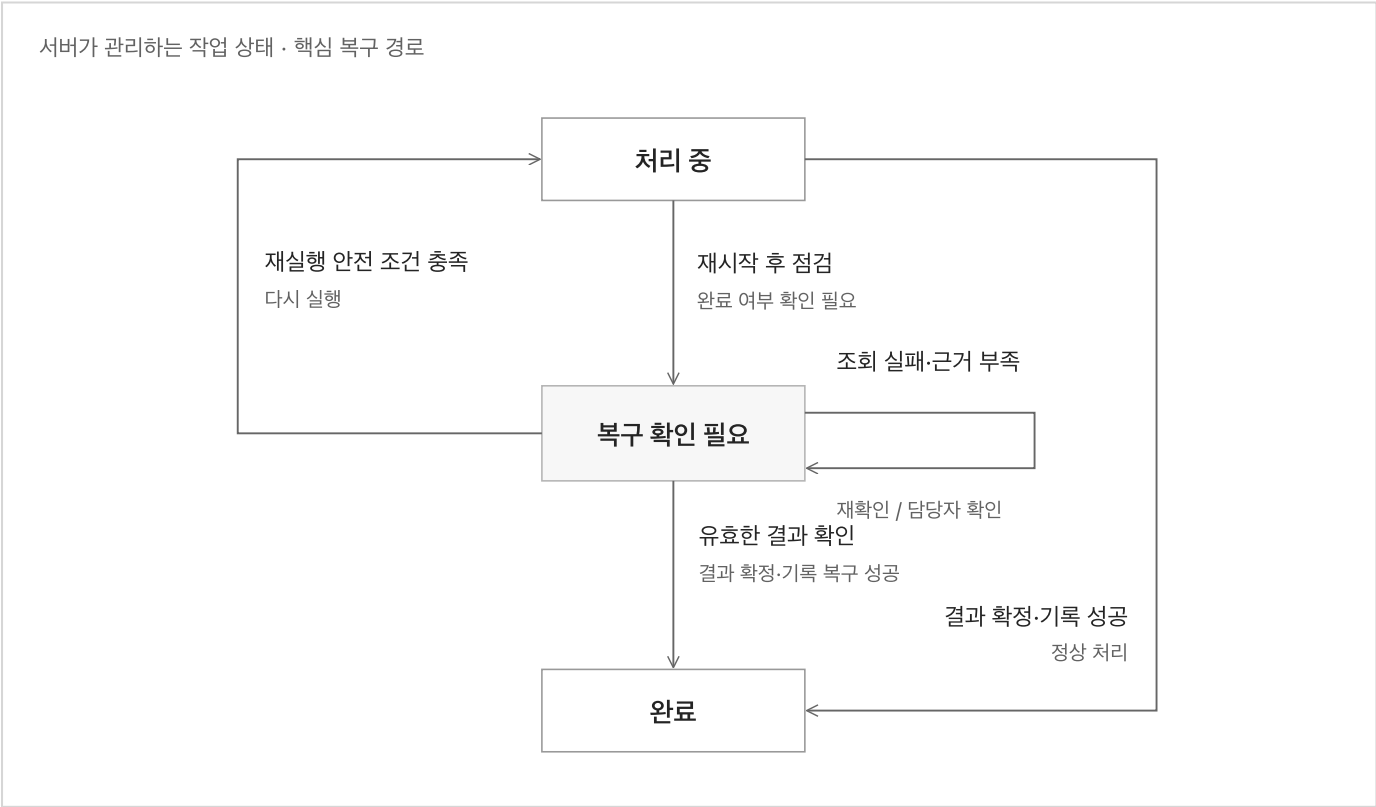
이때 기록만 보고 새 작업을 시작하면 이미 만든 파일을 놓친다. 먼저 작업 ID로 기존 파일을 찾고, 해당 작업의 온전한 결과가 맞는지 확인해야 한다. 맞다면 그 파일을 최종 결과로 확정하고 완료 기록을 복구할 수 있다.

파일이 조회되지 않는다고 곧바로 재실행해도 되는 것은 아니다. 조회 자체가 실패했을 수도 있고, 이전 실행이 아직 진행 중일 수도 있다. 재실행하더라도 최종 결과가 중복으로 확정되지 않는지까지 판단해야 한다.

파일 저장과 DB 기록은 서로 다른 저장소에 있다. DB 트랜잭션 하나만으로 두 작업이 함께 성공한다고 볼 수는 없다.

이제 상태별로 허용할 행동이 달라진다

같은 재요청이라도 처리 중이면 진행 상황을 알려주고, 완료됐으면 기존 결과를 돌려주면 된다. 복구가 필요하면 확인부터 해야 한다. 이 규칙이 여러 곳의 조건문에 흩어지기 시작하면, 상태와 전이를 한곳에서 관리하는 편이 낫다.



클라이언트의 타임아웃이 서버 상태를 바꾸는 것은 아니다. 여기서는 재시작 후 끝나지 않은 작업 기록을 점검하면서 복구를 시작한다.

중요한 건 화살표에 붙은 조건이다. '복구 확인 필요'라는 이름만 추가해서는 다음 행동이 안전해지지 않는다.

기존 결과를 찾았다면 작업 ID와 결과의 유효성을 확인한다. 최종 결과 확정과 완료 기록 복구까지 성공해야 완료로 옮긴다.

다시 실행하려면 이전 실행과 겹치더라도 결과를 하나만 확정하고, 이미 확정된 결과를 바꾸지 않을 수 있어야 한다.

아직 판단 못 한다면 확인 필요 상태를 유지한다. 다음 확인 시점이나 담당자를 정해 작업이 그대로 방치되지 않게 한다.

상태 머신은 이렇게 허용할 행동을 정리한다. 실제로 결과를 하나만 확정하는 일은 저장 방식과 동시 처리 제어가 보장해야 한다. FSM으로 모델링하고 XState 같은 도구로 구현할 수 있지만, 도구를 쓴다고 처리 기록의 저장이나 장애 복구까지 자동으로 해결되지는 않는다.

이 정도면 상태 머신이 필요한가?

단순 조회라면 로딩·성공·오류를 표시하고, 실패했을 때 다시 조회하는 정도로 충분할 수 있다. 상태를 나타내는 값과 한곳의 분기로 대응을 이해할 수 있다면, 별도의 상태 머신 구현을 추가할 이유는 크지 않다.

단순한 상태 값과 분기로 충분	명시적인 상태 머신을 고려
다음 행동이 단순하다 조회 실패 → 다시 조회	현재 상태에 따라 행동이 달라진다 같은 재요청 → 대기 / 결과 반환 / 복구
대응 규칙을 한곳에서 읽을 수 있다	재시도·늦은 응답·복구 규칙이 여러 곳에 흩어진다
실패해도 다시 시도하기 쉽다	잘못된 전이가 중복 결과나 잘못된 완료 안내로 이어진다

지켜야 할 조건을 정했더니

상태별 대응과 복구 규칙이 복잡해지는가?

먼저 무엇을 절대 허용하지 않을지 정하고, 그 조건을 지키는 데 필요한 대응을 적어본다. 그 흐름이 복잡해진다면 상태 머신으로 명시적으로 관리할 때다. 라이브러리 선택은 그다음에 해도 된다.